

Chapter 3

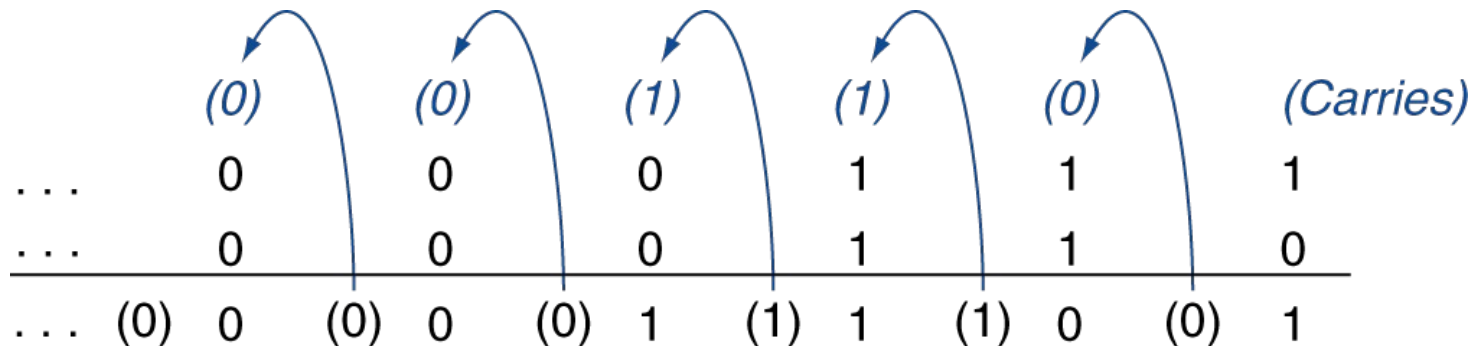
Arithmetic for Computers

3.1 Arithmetic for Computers

- Operations on integers
 - Addition and subtraction
 - Multiplication and division
 - Dealing with overflow
- Floating-point real numbers
 - Representation and operations

3.2 Integer Addition

■ Example: $7 + 6$



■ Overflow if result out of range

- Adding +ve and -ve operands, no overflow
- Adding two +ve operands
 - Overflow if result sign is 1
- Adding two -ve operands
 - Overflow if result sign is 0

3.2 Integer Subtraction

- Add negation of second operand

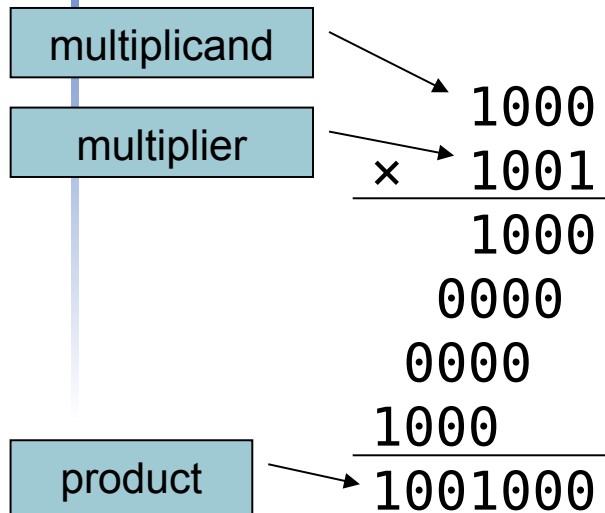
- Example: $7 - 6 = 7 + (-6)$

+7:	0000 0000 ... 0000 0111
<u>-6:</u>	<u>1111 1111 ... 1111 1010</u>
+1:	0000 0000 ... 0000 0001

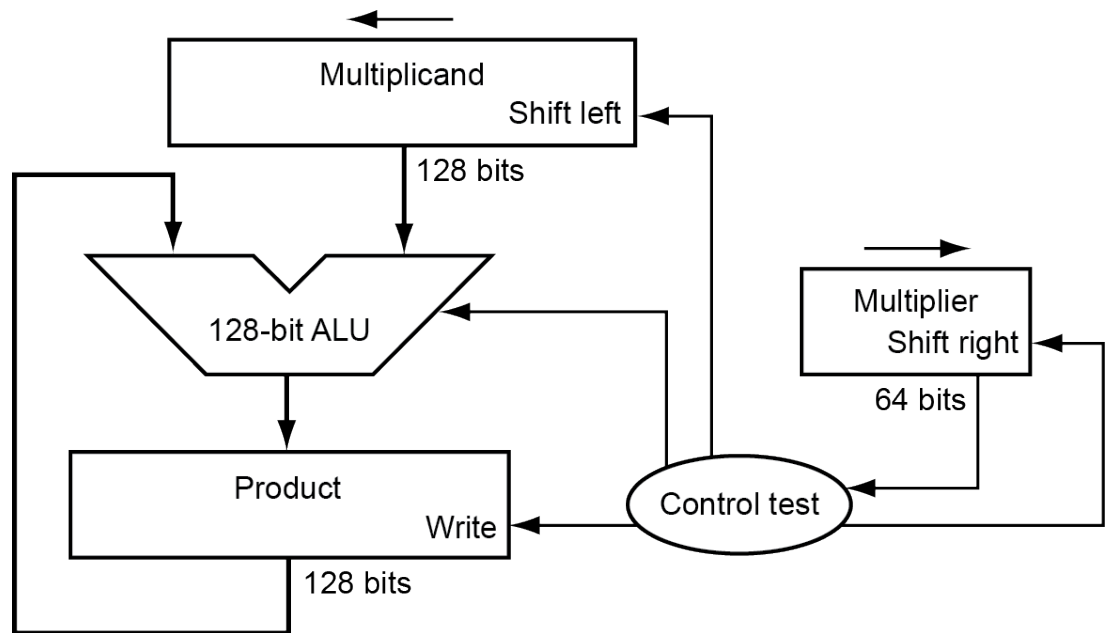
- Overflow if result out of range
 - Subtracting two +ve or two -ve operands, no overflow
 - Subtracting +ve from -ve operand
 - Overflow if result sign is 0
 - Subtracting -ve from +ve operand
 - Overflow if result sign is 1

3.3 Multiplication

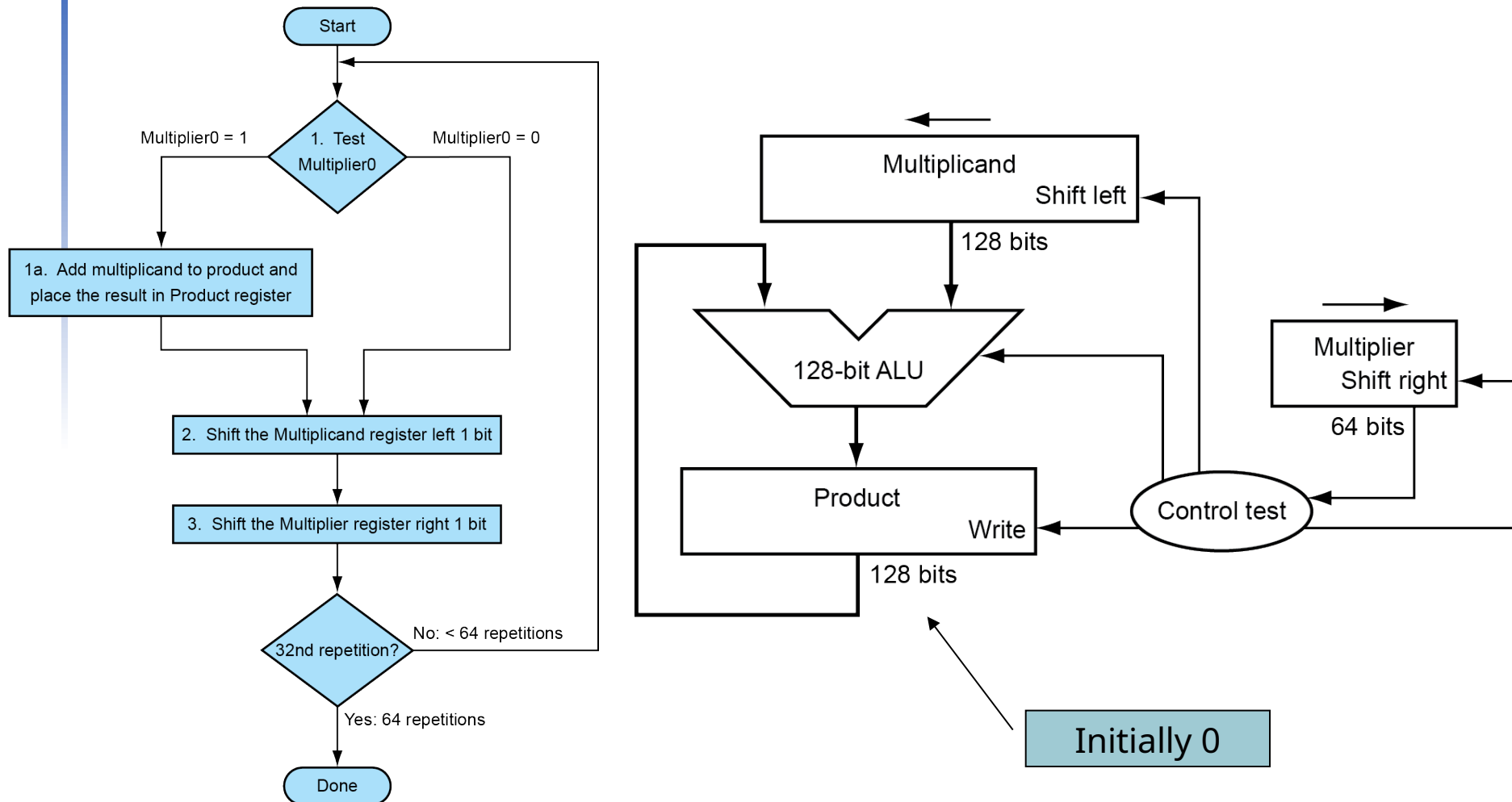
- Start with long-multiplication approach



Length of product is the sum of operand lengths

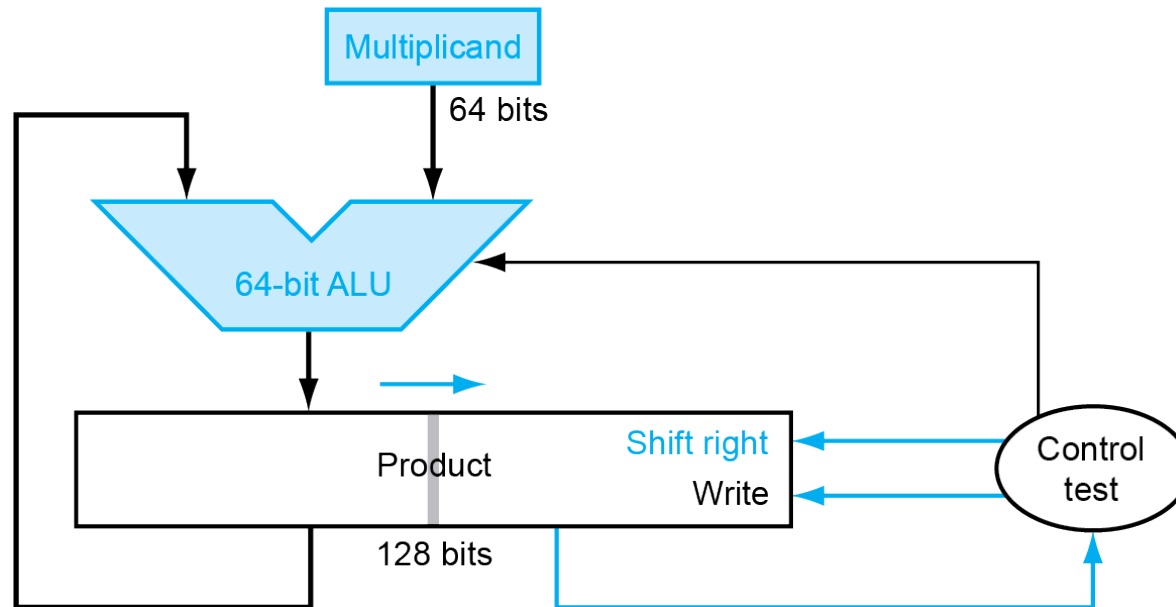


3.3 Multiplication Hardware



3.3 Optimized Multiplier

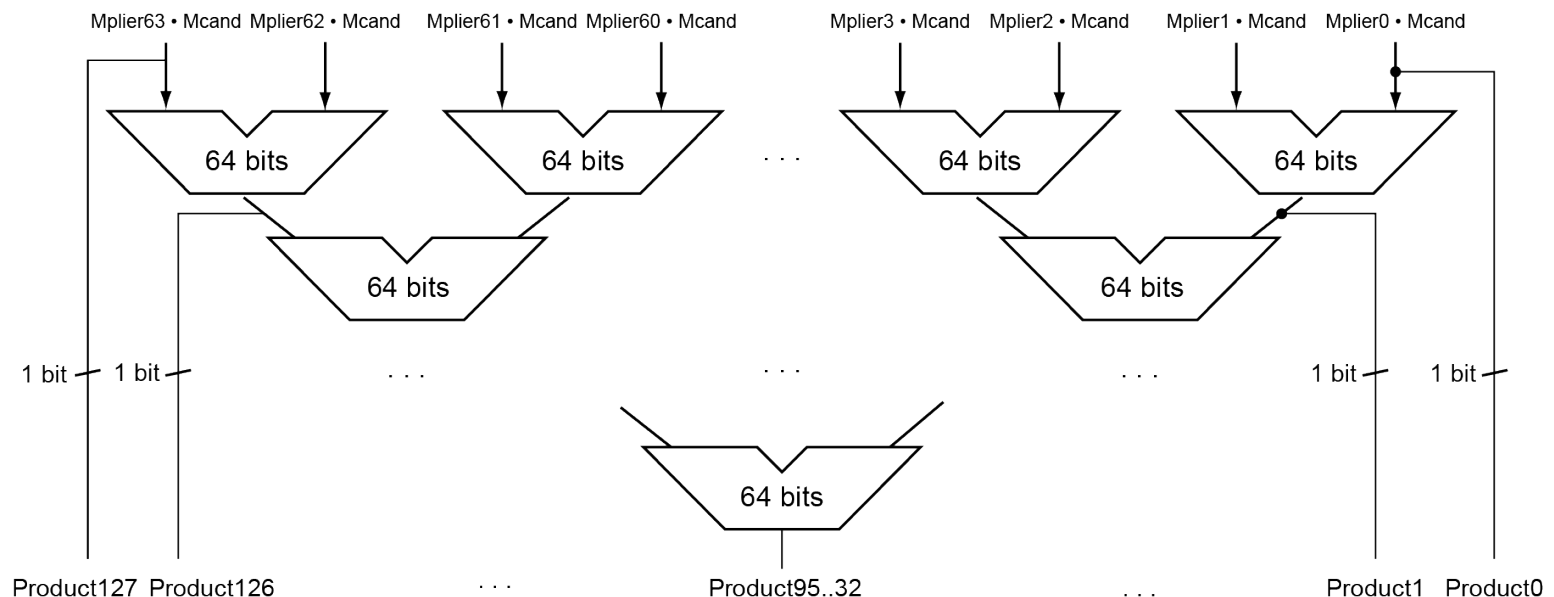
- Perform steps in parallel: add/shift



- One cycle per partial-product addition
 - That's ok, if frequency of multiplications is low

3.3 Faster Multiplier

- Uses multiple adders
 - Cost/performance tradeoff



- Can be pipelined
 - Several multiplication performed in parallel

3.3 RISC-V Multiplication

- Four multiply instructions:
 - `mul`: multiply
 - Gives the lower 64 bits of the product
 - `mulh`: multiply high
 - Gives the upper 64 bits of the product, assuming the operands are signed
 - `mulhu`: multiply high unsigned
 - Gives the upper 64 bits of the product, assuming the operands are unsigned
 - `mulhsu`: multiply high signed/unsigned
 - Gives the upper 64 bits of the product, assuming one operand is signed and the other unsigned
- Use `mulh` result to check for 64-bit overflow